

Darren James Gendron

Senior Golang Software Engineer

+1 (315) 257 6339 • darrengendron56@gmail.com • [Linkedin](#)

Address: 1630 LONGLEAF DR E PINEHURST, NC 28374-8353

Summary

Experienced Backend Engineer specializing in Golang with expertise in Node.js and Python for building scalable microservices, distributed systems, and event-driven architectures. Skilled in designing high-performance APIs, real-time data pipelines, and cloud-native applications using Kafka, gRPC, Kubernetes, and Terraform. Additionally, proficient in React and Angular, contributing to frontend development when needed, ensuring end-to-end system efficiency and seamless user experiences.

Technical Skills

- Programming Languages

- **Primary:** Golang, Typescript, Python, Node.js (JavaScript/TypeScript)
- **Scripting & Automation:** Bash, Shell Scripting

- Backend & API Development

- **Microservices & Distributed Systems:** Golang, Node.js (TypeScript), Python (FastAPI)
- **API Design:** REST, gRPC, GraphQL, WebSockets
- **Authentication & Security:** OAuth 2.0, JWT, OpenID Connect (OIDC), Mutual TLS (mTLS)
- **Concurrency & Performance:** Goroutines, Worker Pools, Channels, Async/Await, Event Loop

- Cloud & Infrastructure

- **Cloud Providers:** AWS, Google Cloud, Azure
- **Serverless & Cloud-Native:** AWS Lambda, Google Cloud Functions
- **Infrastructure as Code (IaC):** Terraform, Pulumi (TypeScript), AWS CDK
- **Containerization & Orchestration:** Docker, Kubernetes (EKS, AKS, GKE), Helm

- DevOps & CI/CD

- **Continuous Integration & Deployment:** GitHub Actions, GitLab CI/CD, Jenkins, ArgoCD, Tekton
- **Deployment Strategies:** Blue-Green Deployments, Canary Releases, Feature Flagging
- **Monitoring & Logging:** Prometheus, Grafana, OpenTelemetry, ELK Stack (Elasticsearch, Logstash, Kibana), Loki, Jaeger, Zipkin

- Database & Storage

- **SQL Databases:** PostgreSQL, MySQL, Amazon Aurora, TimescaleDB
- **NoSQL Databases:** DynamoDB, MongoDB, Cassandra, CockroachDB
- **Caching & Storage:** Redis, Memcached, RocksDB, MinIO, AWS S3

- Messaging & Event-Driven Systems

- **Streaming & Messaging Queues:** Apache Kafka, RabbitMQ, NATS, Pulsar, AWS SNS/SQS
- **Event Processing Frameworks:** Kafka, Streams, Apache Flink, AWS Kinesis

- Version Control & Collaboration

- **Version Control Systems:** Git, GitHub, GitLab, Bitbucket, SVN
- **Project Management & Agile Tools:** Jira, Azure DevOps, Confluence, Trello
- **Code Review & Collaboration:** Pull Requests (PRs), Code Reviews, Pair Programming

- Testing & Quality Assurance

- **Unit Testing:** GoMock (Golang), Jest (TypeScript/Node.js), PyTest (Python)
- **Integration & API Testing:** Postman, Dredd, Supertest (Node.js), TestContainers (Golang, JavaScript)
- **Performance & Load Testing:** K6, Locust, Apache JMeter, wrk
- **Security Testing:** OWASP ZAP, Burp Suite, Trivy (Container Security)

- **End-to-End Testing:** Cypress (TypeScript), Selenium (Python)
- **Security & Compliance**
 - **Application Security:** OWASP Best Practices, API Security Hardening, Role-Based Access Control (RBAC)
 - **Security Tools:** SonarQube, Checkmarx, Trivy, Aqua Security
 - **Secrets Management:** HashiCorp Vault, AWS KMS, Google Secret Manager
 - **Compliance Standards:** PCI-DSS, GDPR, SOC 2, HIPAA
- **Frontend Technologies**
 - **Primary Frontend Stack:** React (Redux, WebSockets), Angular (RxJS, D3.js)
 - **UI Performance Optimization:** Lazy Loading, Change Detection Strategies, Server-Side Rendering (SSR)

Work Experience

- Senior Golang Software Engineer

Fanatics – Pinehurst, NC, United States | 05/2020 – Present

- Architected and developed Golang microservices for the Digital One Omni Channel project, utilizing Goroutines and worker pools for efficient concurrency, gRPC for low-latency service-to-service communication, and Redis caching to handle high-volume transactions across mobile, web, and ATM banking channels.
- Leveraged Go's unique CSP (Communicating Sequential Processes) model with channels and select statements to build a fault-tolerant real-time transaction processing system, reducing synchronization overhead and improving parallel execution of banking operations.
- Designed an event-driven architecture using Apache Kafka and RabbitMQ, implementing idempotent event handling and outbox patterns to ensure eventual consistency across distributed banking microservices while optimizing producer-consumer workflows for peak traffic.
- Implemented streaming data pipelines with Kafka Streams and Redis Streams, optimizing log-based change data capture (CDC) to enable real-time fraud detection, transaction reconciliation, and customer notifications while reducing data synchronization latency.
- Built secure, scalable APIs using Golang, Node.js (TypeScript), and Python (FastAPI), integrating OAuth 2.0, JWT-based authentication, and mTLS encryption with API Gateway-based rate limiting to protect financial data and comply with banking security regulations.
- Optimized PostgreSQL database performance by implementing advanced indexing, query tuning with EXPLAIN ANALYZE, connection pooling with pgBouncer, and partitioning strategies, ensuring optimal performance for large-scale transactional data.
- Developed a high-performance batch processing system in Golang, leveraging worker pools and buffered channels for parallel execution of bulk financial transactions, reducing processing time for end-of-day bank settlements.
- Designed and enforced observability best practices, integrating OpenTelemetry for distributed tracing, Prometheus for metrics collection, and ELK Stack (Elasticsearch, Logstash, Kibana) for centralized logging, enabling real-time monitoring and anomaly detection.
- Implemented structured logging with ZeroLog in Golang and Winston in Node.js, improving log aggregation efficiency and enabling context-aware debugging for production systems with millions of daily transactions.
- Refactored Golang microservices using interfaces and dependency injection (DI) to improve testability, decouple business logic from infrastructure code, and ensure seamless integration with various database backends and external APIs.

- Developed an AI-powered anomaly detection service in Python (FastAPI, NumPy, Pandas), analyzing transaction behaviors using statistical models and integrating it with FIS's real-time fraud detection engine to prevent unauthorized transactions.
- Built a secure financial document storage service using Golang and AWS S3, implementing encryption at rest with AWS KMS and fine-grained IAM permissions to meet banking compliance standards like PCI-DSS and SOC 2.
- Led containerized deployments with Docker and Kubernetes (EKS), implementing Helm charts for deployment automation, horizontal pod autoscaling (HPA), and blue-green deployments with ArgoCD to minimize downtime during releases.
- Automated infrastructure provisioning using Terraform and AWS CDK (TypeScript), defining infrastructure-as-code (IaC) for AWS Lambda, DynamoDB, and API Gateway, reducing manual cloud configurations and improving deployment consistency.
- Implemented Golang's memory optimization techniques, reducing unnecessary heap allocations by leveraging sync.Pool, struct alignment, and escape analysis to improve garbage collection efficiency and reduce memory overhead in high-throughput services.
- Designed an advanced caching layer using Redis and Golang's sync.Map, implementing LRU caching, read-through strategies, and cache invalidation mechanisms to enhance API response times for frequently accessed customer data.
- Developed a WebSockets-based real-time notification service using Node.js (TypeScript) and Golang, enabling instant alerts for transaction approvals, fraud warnings, and system health monitoring, reducing user wait times.
- Built a React-based internal monitoring dashboard, integrating Redux for state management, WebSockets for live updates, and D3.js for interactive financial data visualizations, providing bank administrators real-time insights into transaction performance and API health.
- Extended an Angular-based reporting system, leveraging RxJS for reactive state management and optimizing change detection strategies to ensure smooth rendering of complex financial reports with real-time streaming data.
- Implemented advanced CI/CD pipelines using GitHub Actions and Jenkins, integrating Golang's go test, Jest for TypeScript, and PyTest for Python, enforcing high test coverage and ensuring safe deployments through automated integration testing.

- Software Engineer

Tebra – Pinehurst, NC, United States | 05/2019 – 05/2020

- Developed Golang microservices for real-time credit card payment processing, using worker pools, buffered channels, and mutex locking to optimize parallel transaction handling and reduce processing time during high-traffic periods.
- Designed an event-driven architecture with NATS and AWS SNS/SQS, implementing at-least-once message delivery, dead-letter queues, and distributed transactions to improve resilience and ensure accurate financial settlements.
- Built a Golang-based high-frequency fraud detection system, leveraging bloom filters, probabilistic data structures, and sliding window rate limiting to detect anomalies in card transactions with near-instant processing.
- Implemented a dispute resolution system in Python (FastAPI) and Node.js (TypeScript), integrating chargeback automation with Visa and Mastercard APIs, reducing manual dispute processing time for credit unions.
- Engineered a multi-region ledger system using PostgreSQL with TimescaleDB, applying partitioning and point-in-time recovery to support cross-border transactions and regulatory compliance.
- Developed a Golang-based batch settlement engine, optimizing bulk ACH, wire transfers, and card payment reconciliations with concurrent Goroutines and transaction retry strategies.
- Created an AI-powered risk analysis engine in Python, integrating scikit-learn and real-time transaction streaming with Kafka, flagging suspicious activities based on historical spending patterns and behavioral analytics.

- Implemented an LSM-tree caching strategy using RocksDB to reduce latency in fraud detection queries, replacing traditional Redis caching where persistence and fast lookups were required.
- Designed an API Gateway security layer with mTLS authentication, JWT-based access control, and automated WAF rules, ensuring compliance with PCI-DSS and FFIEC guidelines.
- Integrated Hyperledger Fabric (Blockchain) for immutable financial record-keeping, allowing banks and regulators to audit dispute resolutions, fraud investigations, and compliance reports.
- Automated CI/CD workflows using GitLab CI/CD and Tekton, implementing progressive canary releases and automated rollback mechanisms for backend services in AWS ECS (Fargate).
- Built a React-based fraud monitoring dashboard, integrating Kafka consumer WebSockets for live transaction tracking, providing real-time alerts and visualization of fraudulent activities.
- Extended an Angular-based payment reporting system, using RxJS for live balance updates and D3.js for data visualizations, helping finance teams analyze transaction trends.
- Developed a secure financial document storage system using MinIO and AWS S3, implementing Golang-based client-side encryption for sensitive customer data.
- Designed an advanced rate-limiting mechanism in Golang using token bucket algorithms and sliding log windows, preventing brute-force API attacks and fraudulent transaction bursts.
- Implemented a high-availability financial data ingestion pipeline using Apache Flink and Kafka Streams, processing millions of real-time credit card transactions with low-latency analytics.
- Optimized PostgreSQL query performance for complex payment reconciliation reports, using materialized views, parallel query execution, and JSONB indexing to speed up financial analysis.
- Hardened container security with Trivy and Aqua Security, enforcing strict vulnerability scanning policies in Docker and Kubernetes before deployment to production environments.
- Built an intelligent transaction retry mechanism in Node.js (TypeScript) using exponential backoff and circuit breaker patterns, preventing cascading failures in high-volume API requests.

- Software Engineer

Luxoft – Pinehurst, NC, United States | 09/2017 – 05/2019

- Developed Golang microservices for a real-time financial risk modeling platform, implementing worker pools, Goroutines, and memory-optimized data structures to process millions of market transactions per second.
- Designed and deployed a risk assessment engine using Python (FastAPI) and Golang, integrating Monte Carlo simulations and Bayesian inference to predict portfolio risks for institutional investors.
- Engineered a high-throughput data ingestion pipeline using Apache Kafka and PostgreSQL (TimescaleDB), optimizing historical financial data storage with partitioning, compression, and parallel query execution.
- Implemented secure API integrations with Bloomberg, NASDAQ, and FINRA financial data providers, using OAuth 2.0, WebSockets, and GraphQL to deliver low-latency stock market insights.
- Optimized Go and Node.js performance by tuning Garbage Collection strategies, struct alignment, and CPU profiling (pprof), reducing execution time for computationally expensive financial calculations.
- Developed an event-driven trade settlement system using AWS Lambda, DynamoDB Streams, and SQS, ensuring compliant post-trade processing with FIX protocol integration.
- Implemented a real-time fraud detection module with Python (scikit-learn, TensorFlow), analyzing transaction anomalies and applying time-series forecasting to prevent insider trading violations.

- Designed a secure audit trail system using Hyperledger Fabric (Blockchain), ensuring immutable transaction logs for compliance with SEC and MiFID II regulations.
- Automated financial reporting and reconciliation processes using Golang and Node.js, integrating real-time balance sheet updates with accounting platforms via gRPC and WebSockets.
- Built a serverless financial alerting system with AWS Step Functions and SNS, sending real-time risk exposure notifications based on market volatility indexes (VIX, VAR).
- Developed a WebSocket-based financial dashboard in React and TypeScript, integrating live stock market data feeds, risk exposure analytics, and AI-driven trade recommendations.
- Maintained and optimized an Angular-based regulatory compliance portal, using RxJS for reactive event handling and D3.js for interactive financial data visualizations.
- Designed an LRU-based caching layer in Golang, replacing Redis with a memory-efficient in-memory key-value store, improving low-latency access to pre-computed risk analysis results.
- Deployed services on Kubernetes (AKS) with Helm, leveraging pod auto-scaling, Istio service mesh, and horizontal scaling policies to handle unpredictable market fluctuations.
- Automated infrastructure provisioning with Pulumi (TypeScript), managing multi-cloud deployments across AWS, GCP, and Azure, reducing vendor lock-in risks for financial data processing.
- Implemented a market prediction engine using LSTM neural networks in Python (PyTorch), forecasting stock price trends based on real-time trade sentiment analysis.
- Optimized query performance for complex financial reports, using window functions, CTEs, and parallel index scanning in PostgreSQL, reducing reporting latency for portfolio managers.
- Hardened API security using mutual TLS (mTLS), token-based authentication (JWT), and request signing with HMAC, ensuring end-to-end encryption for financial transactions.
- Established a multi-tier CI/CD pipeline using GitHub Actions, ArgoCD, and Tekton, automating blue-green deployments for fintech applications with zero downtime.

Education

Medaille University(Buffalo, New York)

09/2013-09/2017

Bachelor's Degree in Computer Science

GPA: 3.1

Certifications

Certified Golang Developer	Google
Hashicorp Certified: Terraform Associate	Hashicorp
React Certified Developer	Meta (formerly Facebook)
JavaScript Certification	W3C & Udemy
Certified Web Professional - Web Developer	International Web Association (IWA)
Frontend Web Development Certification	freeCodeCamp
Responsive Web Design Certification	freeCodeCamp
Certified ScrumMaster (CSM)	Scrum Alliance
AWS Certified Developer - Associate	Amazon Web Services
Microsoft Certified: Azure Fundamentals	Microsoft
Google Mobile Web Specialist	Google